Jean Victor Borges de Abreu

Pseudo Aleatoriedade e Segurança Criptográfica: Uma Análise da Transição para Algoritmos Pós-Quânticos

Petrópolis -Rio de Janeiro
2026

Jean Victor Borges de Abreu

Pseudo Aleatoriedade e Segurança Criptográfica: Uma Análise da Transição para Algoritmos Pós-Quânticos

Trabalho de Conclusão de Curso apresentado à

FACULDADE DE EDUCAÇÃO TECNOLÓGICA DO ESTADO DO RIO DE JANEIRO
FAETERJ/PETRÓPOLIS

Orientador: Nome do Orientador

Petrópolis -Rio de Janeiro
2026

1 Resumo

2 Abstract

3 Lista de Figuras

Lista de ilustrações

Figura 1 – Modelo simples de um PRNG com LCG 17

4 Lista de Abreviaturas

ABNT: Associação Brasileira de Normas Técnicas CSPRNG: Cryptographically Secure Pseudo Random Number Generator DRBG: Deterministic Random Bit Generator FAETERJ: Faculdade de Educação Tecnológica do Estado do Rio de Janeiro JEP: JDK Enhancement Proposals KEM: Key Encapsulation Mechanism LCG: Linear Congruential Generator LWE: Learning With Errors ML-KEM: Module-Lattice-Based Key Encapsulation Mechanism NIST: National Institute of Standards and Technology NRBG: Non-deterministic Random Bit Generator PQC: Post-Quantum Cryptography PRNG: Pseudo Random Number Generator QRNG: Quantum Random Number Generator TRNG: True Random Number Generator

Sumário

1 RESUMO 2

2 ABSTRACT 3

3 LISTA DE FIGURAS 4

Lista de ilustrações 4

4 LISTA DE ABREVIATURAS 5

Sumário 5

5 INTRODUÇÃO 8
5.1 Contextualização do Tema 8
5.2 Justificativa/Problema de Pesquisa 8
5.3 Motivação/Hipotese e Ameaca Tecnológica 8
5.4 Objetivos 8
5.5 Metodologia 8

6 FUNDAMENTOS DA ALEATORIEDADE EM COMPUTAÇÃO . 9
6.1 Entropia e Fontes de Incerteza 9
6.2 A Semente (Seed) como Elemento Crítico 10
6.3 Definição de Aleatoriedade e Tipos de Geradores Aleatórios . . . 11
6.4 Avaliação Estatística da Aleatoriedade 12

7 QUEBRA DE CRIPTOGRAFIAS 15
7.1 Visão Geral da Quebra de Criptografias 15
7.2 Falhas Históricas de Aleatoriedade 15
7.3 Vulnerabilidades em Algoritmos Criptográficos 15

8 NIST SP 800-22: TESTES DE ALEATORIEDADE PARA GERADORES DE NÚMEROS PSEUDOALEATÓRIOS 16
8.0.1 Bateria de Testes 17
8.0.1.1 Random 17
8.0.1.2 SegureRandom 18

9 NIST IR 7977 19
9.1 Visão Geral do NIST IR 7977 19

10 NIST IR 8547 20
10.1 Visão Geral do NIST IR 8547 20

11 CRIPTOGRAFIA PÓS-QUÂNTICA (PQC) 21
11.1 Definição e Objetivos da PQC 21
11.2 Paradigmas Matemáticos da PQC 21
11.3 Criptografia Baseada em Reticulados 21
11.4 Learning With Errors (LWE) 21
11.4.1 Implementação Didática em Java (Demonstração) 21
11.5 Padronização Pós-Quântica 21

12 A TRANSIÇÃO PARA PQC NO ECOSSISTEMA JAVA 22
12.1 Suporte à Criptografia Pós-Quântica no Java 22
12.2 Crypto-Agility 22
12.2.1 Implementação Prática: Demo de Crypto-Agility 22
12.3 Comparativo de Modelos 22
12.3.1 Implementação Prática: Benchmark Clássico vs. Kyber 22

13 DISCUSSÃO E ANÁLISE CRÍTICA 23
13.1 Impacto em Desempenho 23
13.2 Barreiras de Adoção 23
13.3 Implicações Sociais e Éticas 23

14 CONCLUSÃO 24
14.1 Síntese dos Resultados 24
14.2 Resposta ao Problema de Pesquisa 24
14.3 Trabalhos Futuros 24

15 BIBLIOGRAFIAS UTEIS 25

16 CONCLUSÃO 27

17 EXEMPLOS PARA USO DA BIBLIOGRÁFIA 28

REFERÊNCIAS 29

5 Introdução

A onipresença da criptografia na infraestrutura digital moderna (internet, sistemas bancários, IoT e comunicações seguras).

5.1 Contextualização do Tema

Textos

5.2 Justificativa/Problema de Pesquisa

O conflito entre o determinismo computacional e a necessidade de aleatoriedade em sistemas criptográficos.

5.3 Motivação/Hipotese e Ameaca Tecnológica

O impacto da computação quântica — especialmente por meio do Algoritmo de Shor — sobre os modelos atuais de segurança e pseudoaleatoriedade.

5.4 Objetivos

Objetivo Geral: Analisar a evolução dos geradores pseudoaleatórios criptograficamente seguros frente à transição para a criptografia pós-quântica. Objetivos Específicos: Estudar PRNGs e CSPRNGs clássicos Analisar o impacto dos algoritmos quânticos Investigar os novos paradigmas de pseudoaleatoriedade baseados em reticulados Avaliar a adoção desses conceitos no ecossistema Java

5.5 Metodologia

Pesquisa bibliográfica, análise documental de padrões criptográficos e estudo prático no ecossistema Java.

6 FUNDAMENTOS DA ALEATORIEDADE EM COMPUTAÇÃO

6.1 Entropia e Fontes de Incerteza

Captação de eventos físicos imprevisíveis como base para sistemas criptográficos seguros.

Entropia é uma medida da aleatoriedade ou imprevisibilidade dos dados. No contexto da criptografia, a entropia é usada para gerar números aleatórios ou chaves que são essenciais para a comunicação segura e a criptografia. Sem uma boa fonte de entropia, os protocolos criptográficos podem se tornar vulneráveis a ataques que exploram a previsibilidade das chaves geradas.

Na maioria dos sistemas operacionais, a entropia é gerada pela coleta de eventos aleatórios de diversas fontes, como interrupções de hardware, movimentos do mouse, pressionamentos de teclas e atividade do disco. Esses eventos são inseridos em um conjunto de entropia, que é então usado para gerar números aleatórios quando necessário.

Protocolos criptográficos, como SSL/TLS, SSH e IPsec, dependem fortemente da entropia para gerar números aleatórios e chaves para criptografia, autenticação e proteção de integridade. A aleatoriedade desses números é crucial para a segurança da comunicação. Se um atacante conseguir prever os números aleatórios usados em um protocolo criptográfico, ele poderá comprometer a confidencialidade, a integridade ou a autenticidade da comunicação. Se a chave mestra prévia for previsível ou tendenciosa, um atacante poderá interceptar a mensagem criptografada, descriptografá-la e obter acesso às chaves de sessão. Isso permitiria ao atacante espionar a comunicação, modificar os dados ou se passar pelo servidor.

Um tipo de entropia. É a entropia de "Shannon". Que utiliza umm tratamento probabilístico de uma função logarítmica.(NASCIMENTO, 2013) Sendo sua formula:

$$\boldsymbol{H(X) = -\sum_{i=1}^{n} P(x_i) \log_b P(x_i)} \tag{6.1}$$

Em que:

- $\boldsymbol{H(X)}$ é a entropia em bits;
- $\boldsymbol{p_i}$ é a probabilidade de ocorrência do resultado $\boldsymbol{i}$;
- $\boldsymbol{n}$ é o número total de resultados possíveis.

Uma limitação determinística é em cima de um processador de maquina de estados finitos. Se acaso for utilizado as mesmas variáveis, o resultado permanece inalterado. Onde

se um invasor tiver a certeza da seed de origem da criptografia. Ela por si mesma, não é aleatória. Para evitar tais acontecimentos se faz necessário que o software faça a captação da incerteza do mundo físico.

A entropia deve ter força superior ou igual a segurança do sistema criptográfico. Entropia adicional poder ser necessária para compensar a perda de entropia durante o processo de geração de chaves, mas não necessariamente adiona mais segurança. Ter mais entropia do que a quantidade avaliada é aceitavel, ter menos do que o recomendado pode ser fatal para a segurança do sistema criptográfico. O comprimento minimo depende do mecanismo criptográfico específico e do nível de segurança desejado. A entrada de entropia e a seed resultante devem ser consistentes com a segurança dos dados protegidos pelo consumidor. Em um modelo DRBG a seed e as chaves geradas deevem possuir o mesmo nivel de segurança, pois no modelo DRBG depende da entropia usada, seguindo o critério de segurança critico (CSP).

6.2 A Semente (Seed) como Elemento Crítico

Falar sobre esse ponto, pois vai ser utilizado na explicação dos protocolos tanto do funcionamento de verificação do NIST 800–22, do NIST IR 8547 e do NIST IR 7977 pos quantico

Uma sequência de bits usada como entrada para um gerador de números pseudoaleatórios (DRBG). A semente é a chave inicial, usada internamente para a concepção do DRBG. Sua entropia deve ser suficiente para garantir a segurança do DRBG. A semente é um elemento crítico, pois determina a sequência de números pseudoaleatórios gerados pelo DRBG. Se a semente for previsível ou tiver baixa entropia, o DRBG pode ser vulnerável a ataques que exploram essa fraqueza. Portanto, é essencial que a semente seja gerada a partir de uma fonte de entropia confiável.

Uma sequência de bits usada como entrada para um mecanismo DRBG. A semente determinará uma parte do interior estado do DRBG, e sua entropia deve ser suficiente para apoiar a força de segurança do DRBG Para obter uma seed suficientemente aleatória. Deve se submeter a uma estimativa de entropia para as saidas da fonte de ruido, baseada na propria fonte da entropia. Essa estimativa é denominada de $\boldsymbol{H}_{\text{submitter}}$. Depois de possuir uma trilha de estimativas, a estimativa de entropia minima por amostra chamada de $\boldsymbol{H}_{\text{original}}$ é determinada. Se o tamanho for superior a 256 bits, ele precisara ser reduzido para menor ou igual a 256 bits.

Se o conjunto de dados não for binário, é necessario uma entropia adicional em bit, denominada de $\boldsymbol{H}_{\text{bitstring}}$ é estimada. O conjunto de dados sequencias possui L amostras, cada uma contendo N bits e é considerado uma cadeia de bits de tamanho $\boldsymbol{L} \times \boldsymbol{N}$. Após os primeiros 1.000.000 bits podem vir a serem desconsiderados entào $\boldsymbol{H}_{\text{bitstring}}$ é calculada. Tendo como amostra estimada como N x $\text{H}_{\text{bitstring}}$. A estimativa da entropia inicial da

fonte é calculada como $\boldsymbol{H}_{\text{initial}} = \mathbf{min}(\boldsymbol{H}_{\text{original}}, \boldsymbol{H}_{\text{bitstring}})$.

6.3 Definição de Aleatoriedade e Tipos de Geradores Aleatórios

RNG (Random Number Generator) é um dispositivo ou algoritmo que gera uma sequência de números ou símbolos que não possuem nenhum padrão discernível, ou seja, são imprevisíveis. Que utiliza uma fonte de incerteza(ou seja, a fonte de entropia) ao mesmo tempo que faz uso de uma função de processo (processo de destilação) para gerar aleatoriedade. O processo de destilação é uma etapa crucial para exceder qualquer ruptura de segurança na origem da entropia que possa a resultar em uma sequencia não aleatória. Como a geração de longas sucessoes de zeros ou uns, ou mesmo a repetição de um mesmo número, o que pode ser um indicativo de uma fonte de entropia comprometida. Normalmente a fonte de entropia é atraves de um processo físico, como o ruído térmico, circuito eletrico, o movimento do mouse ou a digitação do teclado, ou os efeitos quanticos de um semicondutorMultiplas fontes podem ser utilizadas para esse processo. As saidas de um RNG podem vir a ser utilizadas para um pseudo gerador de números aleatórios (PRNG). A saida de um RNG deve seguir certos critérios de aleatoriedade medidos por testes estatísticos, como os definidos pelo NIST 800–22, para garantir que os números gerados sejam adequadamente aleatórios para uso em criptografia e outras aplicações sensíveis à segurança. Para fins de criptografia, é crucial que os números gerados por um RNG sejam verdadeiramente aleatórios. No entanto, algumas das fontes como data/hora são previsiveis, que combinados com outras fontes de entropia, podem mitigar a previsibilidade. E que ainda pode se tornar deficiente ao passar pelos testes estastisticos. A produção de RNGs de alta qualidade se tornam um recurso oneroso. Tornando improdutiva quando se faz necessario de multiplas gerações de RNGs. Por isso, a utilização de PRNGs se torna uma alternativa mais viável, pois eles podem gerar grandes quantidades de números pseudoaleatórios a partir de uma semente inicial.

PRNG (PSeudo-Random Number Generator) é um tipo de gerador de números aleatórios que utiliza um algoritmo determinístico para produzir uma sequência de números que aparenta ser aleatória. Ele faz uso de uma semente (seed) inicial para gerar uma sequência de números pseudoaleatórios. Em momentos que se faz necessário uso de impreviabilidade, a seed se faz necessaria ser imprevisivel e oriundas e suma de um RNG. Por padrão um PRNG requer majoritariamente de um RNG. Como o PRNG é um algoritmo determinístico, por isso ele leva o nome de pseudoaleatório, pois a sequência de números gerada por um PRNG é completamente determinada pela semente inicial. Como a reprodução de um sequencia de números pseudoaleatórios é possível se a semente for conhecida, somente ela se faz necessaria salvar e proteger. De forma cômica, muitos dos valores gerados por um PRNG aparentam sem mais aleatórios do que os números gerados por um RNG. Se sua concepção for bem desenvolvida, cada geração utiliza o valor da anterior por meio de

transformações matemáticas, que acabam introduzindo uma aleatoriedade adicional. Uma série de de transformações podem eliminar autocorrelações e padrões, entre a entrada e a saida.

DRBG (Deterministic Random Bit Generator) é um tipo específico de PRNG que é projetado para ser criptograficamente seguro. Ele produz uma sequencia de bits que a partir de uma seed inicial, determina uma sequencia de bits pseudoaleatórios. A seed para instanciar um DRBG deve ser gerada a partir de uma fonte de entropia confiável, como um RBG (Random Bit Generator), para garantir a segurança do DRBG. Se a seed for protegida e tiver alta entropia, o DRBG pode ser considerado seguro para uso em criptografia. Além da dependência de uma semente de alta entropia, a segurança de um DRBG reside em sua capacidade de manter a Backtracking Resistance e a Prediction Resistance. Conforme o NIST SP 800-90A, esses mecanismos garantem que o comprometimento temporário do estado interno do gerador não resulte na exposição de chaves criptográficas geradas anteriormente ou no colapso da imprevisibilidade dos bits futuros

NRBG (Non-deterministic Random Bit Generator) é um tipo de gerador de números aleatórios que se baseia em processos físicos ou eventos imprevisíveis para gerar números aleatórios. Ao contrário dos DRBGs, que são determinísticos e dependem de uma semente para gerar números pseudoaleatórios, os NRBGs não são determinísticos e não dependem de uma semente. Eles podem usar fontes de entropia, como ruído térmico, fenômenos quânticos ou outros eventos físicos imprevisíveis para gerar números aleatórios.

TRNG (True Random Number Generator) utiliza da entropia de eventos fisicos imprevisíveis para gerar números aleatórios ao inves da utilização de algoritmos determinísticos. É normalmente utilizado para gerar chaves criptográficas, Algoritmo de assinatura digital de curva eliptica[ECDSA] ou chaves MAC simetricas, para armazenar e transmitir dados com alta segurança em protocolos criptograficos como o TLS. As chaves geradas por um TRNG são altamente resistentes a ataques de força bruta, pois a entropia de um TRNG é geralmente muito alta, tornando difícil para um atacante adivinhar ou reproduzir as chaves geradas.

6.4 Avaliação Estatística da Aleatoriedade

Existem diversos testes estastisticos que podem ser utilzados em uma sequencia para avaliar e comparar com uma sequencia verdadeiramente aleatoria. A aleatoriedade é uma caracteristica probabilistica, de forma que é possivel descrever em termos de probabilidade. O resultado dos avaliadores estatisticos, quando baseados em uma sequencia de bits verdadeiramente aleatoria, é denominada de priori É capaz de aplicar uma inifinidade de testes estatísticos para avaliar a aleatoriedade de uma sequência de bits. Cada um avaliando um aspecto diferente da aleatoriedade, sendo a presença ou ausencia de padrões, que sendo identificados seria uma indicação que a sequencia não possui as caracteristicas

aleatórias suficientes. Já que existe uma possibilidade de aplicar uma sequencia infinita de testes, nenhum pode ser considerado completo, sendo necessário estabelecer um conjunto de testes estatísticos padronizados para avaliar a aleatoriedade de sequências de bits. E ainda assim devem ser analisados com cuidado para evitar conclusões errôneas sobre a aleatoriedade de uma sequência de bits.

Uma hipótese nula ($\mathbf{H_0}$) é formulada para cada teste estatístico, que geralmente afirma que a sequência de bits é aleatória. Na outra ponta está a hipótese alternativa ($\mathbf{H_1}$), que afirma que a sequência de bits não é aleatória. O teste estatístico é então aplicado à sequência de bits, e um valor de teste é calculado com base nos resultados do teste. Ao final de cada teste se tem uma decisão a ser tomada, que é se rejeitar ou não a hipótese nula.

Para cada distribuição teorica em cima da hipotese nula ($\mathbf{H_0}$) é determinada por meios matematicos. A partir disso, na distribuição de referencia, um valor critico é determinado longe o suficiente para que a probabilidade de obter um resultado tão extremo 0.9 quanto o observado seja menor do que um nivel de significancia predefinido 0.01.

Na realização, se um valor de um teste exceder o valor crítico, a hipótese nula ($\mathbf{H_0}$) é rejeitada. Se o pressuposto da aleatoriedade for verdadeira o valor final da estatistica téra um probabilidade muito baixa de exceder o valor critico.

Só há duas alternativas de decisão, ou a hipótese nula ($\mathbf{H_0}$) é rejeitada ou não é rejeitada. Se a hipótese nula for rejeitada, isso sugere que a sequência de bits não é aleatória, e que pode haver padrões ou correlações presentes na sequência. Se a hipótese nula ($\mathbf{H_0}$) não for rejeitada, isso sugere que a sequência de bits é aleatória, mas não prova que a sequência é verdadeiramente aleatória.

Erro Tipo 1 ocorre quando a hipótese nula ($\mathbf{H_0}$) é rejeitada, mesmo que seja verdadeira. Isso ocorre quando o teste estatístico indica que a sequência de bits não é aleatória. A probabilidade de um erro do tipo 1 é chamaada de nivel de significancia e é denotada por $\boldsymbol{\alpha}$ Na criptografia valores comuns para $\boldsymbol{\alpha}$ são 0.01 ou 0.001, o que significa que há uma probabilidade de 1% ou 0.1% de rejeitar a hipótese nula ($\mathbf{H_0}$) quando ela é verdadeira. É esperado que 1 a cada 100 sequencias de bits sejam rejeitadas.

Erro do Tipo 2 ocorre quando a hipótese nula ($\mathbf{H_0}$) não é rejeitada, mesmo que seja falsa. Isso ocorre quando o teste estatístico indica que a sequência de bits é aleatória, mesmo que haja padrões ou correlações presentes na sequência. A probabilidade de um erro do tipo 2 é denotada por $\boldsymbol{\beta}$ Não há um valor fixo para $\boldsymbol{\beta}$, pois depende do teste estatístico específico e da natureza dos padrões ou correlações presentes na sequência de bits e que possui uma infinidade ded maneiras de que uma sequencia de bits não seja aleatória.

Uma das principais metas da execução de testes estatísticos é minimizar a probabilidade de cometer erros do tipo 2. Diminuindo a probabilidade de ter um gerador ruim aceitar uma sequência de bits como aleatória. O estatístico de teste é usado para calcular um

P-value que resume a força da evidência contra a hipótese nula.

Se P-value = 1, a sequência parece ter aleatoriedade perfeita.

Se P-value = 0, a sequência parece ser completamente não-aleatória.

Se P-value $\geq \boldsymbol{\alpha}$, a hipótese nula é aceita (a sequência parece ser aleatória).

Se P-value $< \boldsymbol{\alpha}$, a hipótese nula é rejeitada (a sequência não é aleatória).

O NIST SP 800–22 é um conjunto de testes estatísticos amplamente utilizado para avaliar a aleatoriedade de sequências de bits geradas por geradores de números aleatórios. Sendo ao todo 15 testes, cada um projetado para detectar diferentes tipos de padrões ou falhas na aleatoriedade. Esses testes incluem:

1. Teste de Frequência (monobit)
2. Teste de Frequência dentro de um bloco
3. Teste de Sequencias
4. Testes para Maior Sequencia de Uns em um bloco
5. Testes de Posto de Matriz Binária
6. Teste da Transformada Discreta de Fourier (Espectral)
7. Teste de Correspondência de Modelos não Sobrepostos
8. Teste de Correspondência de Modelos Sobrepostos
9. Teste "Estatístico Universal"de Maurer
10. Teste de Complexidade Linear
11. Teste Serial
12. Teste de Entropia Aproximada
13. Teste de Somas Cumulativas (Cusums)
14. Teste de Excursões Aleatórias
15. Teste de Variante de Excursões Aleatórias

7 Quebra de criptografias

7.1 Visão Geral da Quebra de Criptografias

A quebra de criptografia refere-se ao processo de comprometer a segurança de um sistema criptográfico, seja por meio da descoberta de vulnerabilidades, exploração de falhas ou desenvolvimento de técnicas que permitem a recuperação de informações protegidas sem autorização. A quebra pode ocorrer devido a falhas na implementação, fraquezas nos algoritmos ou avanços tecnológicos que tornam os métodos de ataque mais eficazes. A evolução da computação, especialmente com o advento da computação quântica, tem levantado preocupações significativas sobre a segurança dos sistemas criptográficos atuais, destacando a necessidade de desenvolver e adotar algoritmos resistentes a ataques futuros.

7.2 Falhas Históricas de Aleatoriedade

Dual_EC_DRBG, um gerador de números pseudoaleatórios que foi suspeito de conter uma backdoor, permitindo que um atacante previsse os números gerados.

O Bug do Debian OpenSSL (2008): Um desenvolvedor removeu uma linha de código que gerava entropia porque ela causava avisos no depurador. O resultado foi que apenas 32.768 chaves SSH eram possíveis. Um ataque de força bruta levava segundos.

Ataques em Cartões Inteligentes (Taiwan): Milhares de IDs digitais foram gerados com sementes fracas, permitindo que chaves privadas RSA fossem fatoradas a partir das chaves públicas.

7.3 Vulnerabilidades em Algoritmos Criptográficos

Algoritmo de grover. capaz de reduzir a forća das entropias das seed pela metade: https://learn.microsoft.com/es-es/azure/quantum/concepts-grovers Algoritmo de Grover e a Semente: Explique que o ataque de Grover não "quebra"a matemática do AES ou da Seed, mas reduz a busca exaustiva. Se você usa uma semente de 128 bits, para um computador quântico ela tem o esforço de apenas $\mathbf{2^{64}}$. Por isso, o NIST IR 7977 exige o dobro de bits (256).Algoritmo de Shor: Este é o "assassino"do RSA e da Curva Elíptica. Enquanto o Grover reduz a força, o Shor aniquila o RSA completamente. É a razão principal da existência da Criptografia Pós-Quântica (PQC).

Colisões em SHA-1: Cite o projeto SHAttered (Google), que provou na prática que o SHA-1 é inseguro ao gerar dois arquivos PDF diferentes com o mesmo hash.

8 NIST SP 800-22: Testes de Aleatoriedade para Geradores de Números Pseudoaleatórios

Bateria de testes SP 800–22 do NIST e o conceito de indistinguibilidade estatística.

O NIST Test Suite consiste em um pacote de 15 testes de estatistas desenvolvidos para avaliar a as sequencias binarias aleatórias (com comprimento arbitrário), gerados por geradores de números aleatórios ou pseudo criptografados, seja de forma em hardware ou software. les podem ser decompostos em vários subtestes de acordo com a necessidade e o tipo de geração aleatória que podem ter uma sequencia. Os 15 testes são:

1. Teste de Frequência (monobit)
2. Teste de Frequência dentro de um bloco
3. Teste de Sequencias
4. Testes para Maior Sequencia de Uns em um bloco
5. Testes de Posto de Matriz Binária
6. Teste da Transformada Discreta de Fourier (Espectral)
7. Teste de Correspondência de Modelos não Sobrepostos
8. Teste de Correspondência de Modelos Sobrepostos
9. Teste "Estatístico Universal"de Maurer
10. Teste de Complexidade Linear
11. Teste Serial
12. Teste de Entropia Aproximada
13. Teste de Somas Cumulativas (Cusums)
14. Teste de Excursões Aleatórias
15. Teste de Variante de Excursões Aleatórias

Falar separadamente sobre cada um dos 15 testes. Mostrando um teste local separado. (Seguir documentacao do guthub para execucao local)(GITHUB, s.d.)

8.0.1 Bateria de Testes

Os arquivos binários gerados em Java servem como a entrada bruta para o NIST SP 800–22, que interpreta os bytes como fluxos contínuos de bits para análise estatística. Ao executar o programa, a ferramenta particiona esses dados em blocos de tamanho fixo, como sequências de $\mathbf{10^6}$ bits, aplicando sobre cada um deles quinze testes que buscam padrões não aleatórios, como repetições excessivas ou falta de complexidade linear. Para cada teste, o sistema calcula um $\boldsymbol{P}$-value, que quantifica a probabilidade de a sequência ter sido gerada por um processo puramente aleatório. Ao final, o sucesso do gerador é determinado pela uniformidade desses valores e pela proporção de sequências que mantiveram o $\boldsymbol{P}$-value $\geq \mathbf{0,01}$.

Figura 1 – Modelo simples de um PRNG com LCG

```java
import java.io.BufferedOutputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.security.SecureRandom;
import java.util.Random;

public class Main {
Run | Debug
public static void main(String[] args) {
        int tamanhoBytes = 12_500_000;

        gerarInseguro(nomeArquivo: "inseguro.bin", tamanhoBytes);
        gerarSeguro(nomeArquivo: "seguro.bin", tamanhoBytes);
    }

    public static void gerarInseguro(String nomeArquivo, int tamanho) {
        Random prng = new Random();
        byte[] buffer = new byte[1024];

        try (BufferedOutputStream bos = new BufferedOutputStream(new FileOutputStream(nomeArquivo))) {
            for (int i = 0; i < tamanho; i += buffer.length) {
                prng.nextBytes(buffer);
                bos.write(buffer);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    public static void gerarSeguro(String nomeArquivo, int tamanho) {
        SecureRandom csprng = new SecureRandom();
        byte[] buffer = new byte[1024];

        try (BufferedOutputStream bos = new BufferedOutputStream(new FileOutputStream(nomeArquivo))) {
            for (int i = 0; i < tamanho; i += buffer.length) {
                csprng.nextBytes(buffer);
                bos.write(buffer);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Fonte: Elaborado pelo autor (2026)

8.0.1.1 Random

A função java.util.Random faz uso do modelo LCG "Gerador Congruencial Linear". Ele é um modelo puramente matemático. Onde o mesmo é usado para gerar uma sequência de números aleatórios, calculados por uma equação linear.

$$\boldsymbol{x_{i+1} = (A \cdot x_i + C) \pmod{M}} \tag{8.1}$$

Onde o valor inicial "x0" é a seed, A e C são parâmetros inteiros que podem ter um valor já pré-definido ou passados diretamente, M é o espaço dos valores aleatórios possíveis e "mod" é o restante da divisão inteira.

8.0.1.2 SegureRandom

A função java.security.SegureRandom faz utilização do CSPRNG, que diferente do LCG. Esse tipo de gerador busca caos do hardware. Sendo elas interrupçoes de hardware, pressionar das teclas do teclado, posição e movimento do mouse, pequenas flutuaçoes de voltagem do processador e/ou o tempo que o disco demora pra fazer a requisição. Esses eventos imprevisíveis são misturados, usando funçoes hash como SHA-256 para criar uma seed de alta entropia. Mesmo que se conheça o algoritmo, não é possivel recriar o mesmo evento. Justamente pelo fator caos fisico que foi utilizado.

9 NIST IR 7977

9.1 Visão Geral do NIST IR 7977

This document describes the principles, processes and procedures that drive cryptographic standards and guidelines development efforts at the National Institute of Standards and Technology (NIST). This document reflects public comments received on two earlier versions, and will serve as the basis to guide NIST's future cryptographic standards and guidelines development efforts. It will be reviewed and updated every five years, or more frequently if a need arises, to help ensure that NIST fulfills its role and responsibilities for producing robust, effective cryptographic standards and guidelines.

10 NIST IR 8547

10.1 Visão Geral do NIST IR 8547

This report describes NIST's expected approach to transitioning from quantum-vulnerable cryptographic algorithms to post-quantum digital signature algorithms and key-establishment schemes. It identifies existing quantum-vulnerable cryptographic standards and the current quantum-resistant standards that will be used in the migration. This report should inform the efforts and timelines of federal agencies, industry, and standards organizations for migrating information technology products, services, and infrastructure to PQC. Comments received on this draft will be used to revise this transition plan and feed into other algorithm- and application-specific guidance for the transition to PQC.

11 CRIPTOGRAFIA PÓS-QUÂNTICA (PQC)

11.1 Definição e Objetivos da PQC

Algoritmos clássicos projetados para resistir a ataques quânticos.

11.2 Paradigmas Matemáticos da PQC

Reticulados, códigos corretores de erro, funções hash e isogenias.

11.3 Criptografia Baseada em Reticulados

Introdução ao conceito de Lattices como base da nova segurança criptográfica.

11.4 Learning With Errors (LWE)

Fundamento matemático da pseudoaleatoriedade pós-quântica.

11.4.1 Implementação Didática em Java (Demonstração)

Implementação simplificada do LWE do zero em Java para fins didáticos, mostrando como o erro intencional funciona matematicamente antes de utilizar a biblioteca pronta.

11.5 Padronização Pós-Quântica

O papel do ML-KEM (Kyber) e o processo de padronização conduzido pelo NIST.

12 A TRANSIÇÃO PARA PQC NO ECOSSISTEMA JAVA

12.1 Suporte à Criptografia Pós-Quântica no Java

Evolução do JDK e integração de algoritmos pós-quânticos em versões modernas (Java 21+).

12.2 Crypto-Agility

Boas práticas para desenvolvimento de sistemas criptográficos flexíveis e preparados para mudanças de algoritmo.

12.2.1 Implementação Prática: Demo de Crypto-Agility

Criação de uma interface Java que abstrai o algoritmo de geração de chaves, permitindo trocar de RSA para Kyber alterando apenas a configuração — sem reescrever o código da aplicação.

12.3 Comparativo de Modelos

Geração de chaves e aleatoriedade no modelo clássico versus modelo baseado em reticulados (Module-Lattice).

12.3.1 Implementação Prática: Benchmark Clássico vs. Kyber

Benchmark comparativo entre java.security.SecureRandom clássico e ML-KEM (Kyber) via Bouncy Castle, medindo tempo de geração de chaves, uso de memória e throughput. Inclui comparativo de tamanho de chaves entre RSA-2048 e Kyber. Nota: Confirmar com o orientador o uso da biblioteca Bouncy Castle como dependência, por ser a opção mais acessível para PQC em Java atualmente.

13 DISCUSSÃO E ANÁLISE CRÍTICA

13.1 Impacto em Desempenho

Custos computacionais, consumo de memória e latência — baseados nos resultados do benchmark da seção 6.3.1.

13.2 Barreiras de Adoção

Sistemas legados, limitações de hardware e resistência organizacional.

13.3 Implicações Sociais e Éticas

Soberania de dados, privacidade, vigilância e assimetria tecnológica.

14 CONCLUSÃO

14.1 Síntese dos Resultados

A inevitabilidade da transição criptográfica.

14.2 Resposta ao Problema de Pesquisa

A evolução da pseudoaleatoriedade como pilar da segurança pós-quântica.

14.3 Trabalhos Futuros

Geradores Quânticos de Números Aleatórios (QRNG) e integração híbrida TRNG + PQC.

15 Bibliografias Uteis

1. Fundamentos e Estado da Arte (Capítulos 2 e 3)

Para falar de aleatoriedade, sementes e o funcionamento dos CSPRNGs atuais:

Livro: The Art of Computer Programming, Vol 2: Seminumerical Algorithms (Donald Knuth).

Onde usar: Capítulos 2.1 e 3.1. É a base matemática para entender geradores lineares e por que eles são previsíveis.

Livro: Handbook of Applied Cryptography (Menezes, van Oorschot e Vanstone).

Onde usar: Capítulo 3.2 e 4.1. Consulte o Capítulo 5 (Pseudorandom Bits and Sequences). Ele define formalmente o "Next Bit Test".

Norma Técnica: NIST SP 800-22 Rev 1a: A Statistical Test Suite for Random and Pseudorandom Number Generators.

Onde usar: Capítulo 2.4. É o documento que dita como o mundo testa se um PRNG é bom.

2. A Ameaça Quântica (Capítulo 4)

Aqui você precisa citar os fundamentos da física quântica aplicados à quebra de códigos:

Livro: Quantum Computation and Quantum Information (Michael Nielsen e Isaac Chuang).

Onde usar: Capítulo 4.1 e 4.2. É a "bíblia"da computação quântica. Essencial para explicar Qubits e o Algoritmo de Shor.

Artigo Seminal: Algorithms for quantum computation: discrete logarithms and factoring (Peter Shor, 1994).

Onde usar: Capítulo 4.2. O artigo que mudou a história da criptografia.

Relatório: NIST IR 8105: Report on Post-Quantum Cryptography.

Onde usar: Capítulo 4.4. Explica o conceito de "Harvest Now, Decrypt Later".

3. Criptografia Pós-Quântica e Reticulados (Capítulo 5)

Esta é a parte mais densa e moderna do seu trabalho:

Artigo de Revisão: A Decade of Lattice Cryptography (Chris Peikert). Onde usar: Capítulo 5.3 e 5.4. Peikert é um dos maiores nomes da área. Este artigo resume como a matemática de reticulados (Lattices) se tornou segura.

Artigo Técnico: The Learning with Errors Problem (Oded Regev). Onde usar: Capítulo 5.4. Oded Regev criou o conceito de LWE. Este artigo é a fonte primária para a sua fundamentação matemática.

Documento Oficial (FIPS): FIPS 203: Module–Lattice–Based Key–Encapsulation Mechanism Standard.

Onde usar: Capítulo 5.5. Este é o padrão final do ML-KEM (Kyber). É a referência mais importante para o seu futuro capítulo 6.

4. Transição e Ecossistema Java (Capítulo 6)

Para a parte teórica sobre Java e Agilidade Criptográfica: Documentação JEP (JDK Enhancement Proposals):

JEP 466 (Class-File API) e documentos relacionados à implementação do ML-KEM no provedor SunJCE.

Onde usar: Capítulo 6.1.

Artigo sobre Crypto-Agility: A Taxonomy of Crypto-Agility (S. Ott et al.).

Onde usar: Capítulo 6.2. Ajuda a definir por que sistemas modernos não podem ficar "presos"a um único algoritmo.

16 Conclusão

Considerações finais.

17 Exemplos para uso da bibliográfia

para livros(KNUTH, 1997)

para artigos(MATSUMOTO; NISHIMURA, 1998)

para sites e outras fontes(Oracle Corporation, 2023)

para outros TCC e afins(ALMEIDA, 2025)

para fichas tecnicas e documentacao(International Organization for Standardization, 2022)

Referências

ALMEIDA, C. *Análise de Geradores de Números Aleatórios em Sistemas Embarcados*. Dissertação (Dissertação (Mestrado em Engenharia Eletrônica)) — Universidade Estadual do Rio de Janeiro, Rio de Janeiro, 2025.

GITHUB. *NIST*. s.d. Disponível em: <https://github.com/terrillmoore/NIST-Statistical-Test-Suite>. Acesso em: 02 mar. 2026.

International Organization for Standardization. *ISO/IEC 27001: Information technology - Security techniques*. Geneva, CH, 2022.

KNUTH, D. E. *The Art of Computer Programming: Seminumerical Algorithms*. Reading, MA: Addison-Wesley, 1997. v. 2.

MATSUMOTO, M.; NISHIMURA, T. Mersenne twister: A 623-dimensionally equidistributed uniform pseudo-random number generator. *ACM Transactions on Modeling and Computer Simulation*, v. 8, n. 1, p. 3–30, 1998.

NASCIMENTO, W. S. *Sobre algumas características da entropia de Shannon para sistemas atômicos confinados*. Dissertação (Dissertação (Mestrado em Física)) — Universidade Federal da Bahia Instituto de Física, Bahia, 2013. Disponível em: <https://repositorio.ufba.br/bitstream/ri/28664/1/disserta%c3%a7%c3%a3o_wallas_final.pdf>. Acesso em: 25 fev. 2026.

NETDATA. *Understanding Entropy Key To Secure Cryptography & Randomness*. s.d. Disponível em: <https://www.netdata.cloud/blog/understanding-entropy-the-key-to-secure-cryptography-and-randomness>. Acesso em: 18 fev. 2026.

Oracle Corporation. *Class Random: Java Platform SE 17*. 2023. Disponível em: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Random.html>. Acesso em: 18 fev. 2026.

RUKHIN, A. A statistical test suite for random and pseudorandom number generators for cryptographic applications. *National Institute of Standards and Technology*, v. 2, n. 1, p. 23–62, 2010.